

Live Streaming & Social Platform MVP — Full Product & Technical PRD

Version: 1.0

Status: Implementation Baseline

Source: Technical Assessment requirement document

Stack decision: Next.js + Node.js/Express + PostgreSQL + Redis

Docker: Not used for the initial implementation

1. Executive Summary

Build a production-quality foundation for a social live-streaming platform inspired by products such as Tango.

The MVP must allow users to:

- Register and authenticate.
- Create and update profiles.
- Follow/unfollow other users.
- Start and end live streams.
- Watch active streams.
- See viewer counts.
- Participate in real-time chat.
- Send mock virtual gifts using virtual credits.
- View creator profiles and activity.

Administrators must be able to:

- View users.
- View active streams.
- View reports.
- Disable users.
- Terminate active streams.

The assessment explicitly evaluates architecture, database design, real-time functionality, security, API design, code quality, testing, and documentation. The implementation must therefore prioritize correctness, transactional safety, authorization, failure handling, and maintainability over feature quantity.

2. Goals

2.1 Primary Goals

1. Build the complete assessment MVP.
2. Preserve the assessment's required endpoint names wherever possible.
3. Use Next.js for the frontend.
4. Use Node.js + Express + TypeScript for the backend.
5. Use PostgreSQL as the source of truth for persistent data.
6. Use Redis for ephemeral state and real-time infrastructure.
7. Implement real-time chat with Socket.IO.
8. Implement live video using WebRTC with a self-hosted/open-source media layer for the initial implementation.
9. Make all critical state-changing operations atomic and rollback-safe.
10. Prevent unauthorized access and client-side manipulation.
11. Provide automated tests, migrations, seeds, API documentation, and architecture documentation.
12. Keep the architecture ready for future scaling without prematurely introducing microservices.

2.2 Non-Goals

The MVP does not need:

- Real-money payments.
- Video recording/playback.
- Large-scale CDN infrastructure.
- 100,000-user production capacity.
- Advanced AI moderation.
- Full recommendation systems.
- Complex notification systems.
- Multiple-guest broadcasting unless time permits.
- Docker during the initial development phase.

The source requirement explicitly states that the assessment is not intended to be the entire platform and recommends a 12–20 hour assessment duration.

3. Technology Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

- TanStack Query
- Socket.IO Client
- WebRTC APIs
- React Hook Form
- Zod

Backend

- Node.js
- Express
- TypeScript
- Prisma ORM
- PostgreSQL
- Redis
- Socket.IO
- JWT authentication
- Swagger/OpenAPI

Testing

- Jest
- Supertest
- React Testing Library where appropriate

Development

- npm/pnpm
- ESLint
- Prettier
- Git hooks where useful

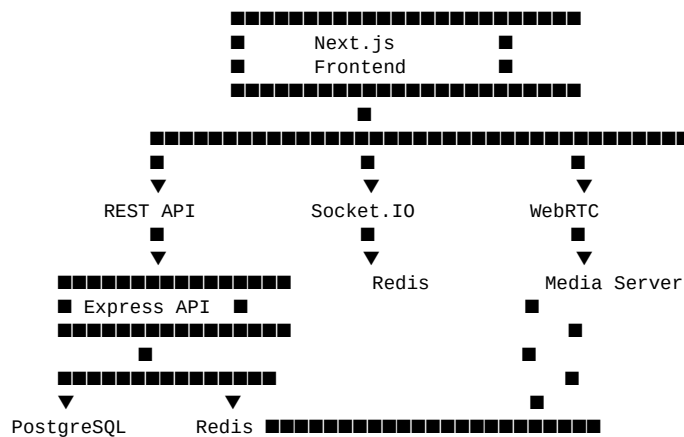
Streaming

- WebRTC
- MediaMTX or equivalent open-source media server

Docker is explicitly excluded from the first implementation phase.

4. Architecture

4.1 High-Level Architecture



4.2 Responsibilities

Next.js

Responsible for:

- UI.
- Routing.
- Authentication state.
- REST API consumption.
- WebSocket connection.
- WebRTC client logic.
- Client-side validation.
- User experience.

Express

Responsible for:

- Authentication.
- Authorization.
- API routing.
- Validation.
- Business logic.
- Transactions.
- Database operations.
- Stream lifecycle.
- Wallet/gift operations.
- Admin operations.

PostgreSQL

Authoritative persistent source of truth for:

- Users.
- Profiles.
- Follows.
- Streams.
- Stream viewers/history.
- Chat messages.
- Gifts.
- Wallets.
- Gift transactions.
- Reports.

Redis

Used for:

- Viewer presence.
- WebSocket scaling.
- Rate limiting.
- Temporary state.
- Heartbeats/TTL.
- Distributed coordination where necessary.

Redis must not replace PostgreSQL as the source of truth for financial-style operations.

Media Server

Responsible for:

- Live media transport.
- WebRTC publishing.
- WebRTC playback.

The application backend must not proxy video traffic.

5. Architecture Style

Use a **modular monolith**.

Do not create unnecessary microservices.

Backend modules:

```
auth
users
profiles
follows
streams
chat
wallet
gifts
reports
admin
health
```

Each module should have clear boundaries:

```
routes/controllers
services
repositories
validators
DTOs/types
tests
```

Controllers must remain thin.

Business rules belong in services.

Database access belongs in repositories/data-access layers.

6. Project Structure

```
live-platform/
├── apps/
│   ├── web/
│   │   ├── app/
│   │   │   ├── components/
│   │   │   ├── features/
│   │   │   ├── hooks/
│   │   │   ├── lib/
│   │   │   ├── types/
│   │   │   └── utils/
│   │   └── api/
│   │       └── src/
│   │           ├── config/
│   │           ├── middleware/
│   │           ├── modules/
│   │           │   ├── auth/
│   │           │   ├── users/
│   │           │   ├── profiles/
│   │           │   ├── follows/
│   │           │   ├── streams/
│   │           │   ├── chat/
│   │           │   ├── wallet/
│   │           │   ├── gifts/
│   │           │   ├── reports/
│   │           │   ├── admin/
│   │           │   └── health/
│   │           ├── infrastructure/
│   │           ├── utils/
│   │           ├── app.ts
│   │           └── server.ts
│   └── packages/
│       ├── shared/
│       └── prisma/
│           ├── schema.prisma
│           ├── migrations/
│           └── seed.ts
```

```
■
■■■ docs/
■   ■■■ architecture.md
■   ■■■ database.md
■   ■■■ streaming.md
■   ■■■ security.md
■
■■■ .env.example
■■■ README.md
■■■ package.json
```

7. Environment Configuration

Required configuration:

```
NODE_ENV=development

PORT=4000

DATABASE_URL=

REDIS_URL=

JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=

ACCESS_TOKEN_TTL=
REFRESH_TOKEN_TTL=

CORS_ORIGIN=

STREAMING_SERVER_URL=
STREAMING_PUBLIC_URL=
STREAMING_API_SECRET=

RATE_LIMIT_WINDOW_MS=
RATE_LIMIT_MAX_REQUESTS=
```

Rules:

- Never commit `.env`.
- Commit `.env.example`.
- Secrets must never be returned in API responses.
- Secrets must never be logged.
- Startup should fail fast if mandatory environment variables are missing.
- Production secrets must be supplied by the deployment environment.

8. API Conventions

Base API:

```
/api/v1
```

For strict assessment compatibility, the required endpoint names should remain recognizable.

Example:

```
POST /api/v1/auth/register
```

```
POST /api/v1/auth/login
GET /api/v1/users/me
```

All API responses should use a consistent structure.

Success

```
{
  "success": true,
  "data": {}
}
```

Error

```
{
  "success": false,
  "error": {
    "code": "STREAM_NOT_FOUND",
    "message": "Stream not found",
    "requestId": "req_123"
  }
}
```

Never expose stack traces to clients.

9. Authentication

9.1 Registration

```
POST /auth/register
```

Input:

```
{
  "username": "grey",
  "email": "grey@example.com",
  "password": "StrongPassword123"
}
```

Process:

```
Validate input
↓
Normalize email/username
↓
Check uniqueness
↓
Hash password
↓
BEGIN TRANSACTION
↓
Create user
↓
Create profile
↓
Create wallet
↓
COMMIT
↓
Issue authentication tokens
```

If any database operation fails:

ROLLBACK

No partially-created account may remain.

9.2 Password Security

Passwords must:

- Never be stored plaintext.
- Never be logged.
- Be hashed using a modern password hashing algorithm such as Argon2id or bcrypt with an appropriate cost.
- Never be returned from an API.

10. Token Architecture

Use:

```
Short-lived access token
+
rotating refresh token
```

Access token:

```
~15 minutes
```

Refresh token:

```
Longer-lived
```

For browser use:

- Refresh token should use an HttpOnly, Secure cookie.
- SameSite should be appropriately configured.
- Access tokens should not be unnecessarily persisted in localStorage.

Refresh-token rotation must detect token reuse.

11. Logout

```
POST /auth/logout
```

Invalidate the refresh-token/session record.

Logout must be idempotent.

Calling logout twice must not cause an error.

12. Authorization

Roles:

```
USER
ADMIN
```

Middleware:

```
authenticate
authorize(role)
```

Authorization must be checked on the server.

Never trust:

```
userId
role
balance
giftPrice
stream ownership
```

sent by the frontend.

The authenticated identity comes from the verified token/session.

13. User APIs

Required:

```
POST /auth/register
POST /auth/login
GET /users/me
GET /users/:id
PUT /users/me
POST /users/:id/follow
DELETE /users/:id/follow
```

14. User Database

```
users
```

Fields:

```
id UUID PRIMARY KEY
username VARCHAR UNIQUE
email VARCHAR UNIQUE
password_hash VARCHAR
role ENUM
status ENUM
created_at TIMESTAMP
updated_at TIMESTAMP
last_login_at TIMESTAMP NULL
```

Indexes:

```
username
email
status
created_at
```

15. Profile Database

```
profiles
```

Fields:

```
id UUID
user_id UUID UNIQUE
display_name
profile_picture_url
bio
followers_count
following_count
created_at
updated_at
```

Counts are denormalized for efficient reads but must be updated safely.

For operations affecting counts:

```
BEGIN
modify follow relationship
modify count
COMMIT
```

If the transaction fails, both changes roll back.

16. Follow System

```
POST /users/:id/follow
DELETE /users/:id/follow
```

Rules:

- Cannot follow yourself.
- Cannot create duplicate follows.
- Unfollow nonexistent relationship should be safely idempotent.
- Disabled users cannot be followed.
- Counts must remain consistent.

Database:

```
follows
```

```
id
follower_id
following_id
created_at
```

Constraint:

```
UNIQUE(follower_id, following_id)
```

17. Streaming

Required endpoints:

```
POST /streams
GET /streams
GET /streams/live
GET /streams/:id
POST /streams/:id/join
POST /streams/:id/end
```

These match the assessment requirements.

18. Stream Database

```
streams
```

Fields:

```
id UUID
broadcaster_id UUID
title VARCHAR
status ENUM
stream_key_hash VARCHAR
viewer_count INTEGER
started_at TIMESTAMP NULL
ended_at TIMESTAMP NULL
created_at TIMESTAMP
updated_at TIMESTAMP
```

Statuses:

```
SCHEDULED
LIVE
ENDED
```

19. Stream State Machine

Allowed:

```
SCHEDULED → LIVE
LIVE → ENDED
```

Not allowed:

```
ENDED → LIVE
ENDED → SCHEDULED
LIVE → SCHEDULED
```

Every transition must be validated by the backend.

20. Create Stream

POST /streams

Input:

```
{
  "title": "Late Night Stream"
}
```

Rules:

- User must be authenticated.
- User must be ACTIVE.
- User cannot already have another LIVE stream.
- Title must be validated.
- Stream ID generated server-side.
- Streaming credentials generated server-side.

Database transaction:

```
BEGIN
↓
verify user
↓
verify no active stream
↓
create stream
↓
create required stream state
↓
COMMIT
```

Failure:

```
ROLLBACK
```

21. Start Streaming

The API/database lifecycle and media lifecycle must not be treated as the same operation.

Flow:

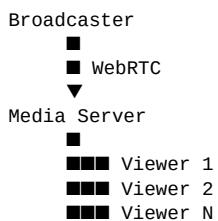
```
POST /streams
↓
SCHEDULED
↓
obtain publish credentials
↓
connect to media server
↓
media publish confirmed
↓
stream becomes LIVE
```

Do not mark a stream LIVE merely because a database row was created.

The media layer must be reachable/confirmed.

22. Media Architecture

Initial implementation:



Application server:

does NOT transport video

It manages:

stream ownership
stream metadata
authorization
stream state
viewer state

23. Stream Viewer Join

POST /streams/:id/join

Process:

```
Authenticate
↓
Check user status
↓
Check stream exists
↓
Check status = LIVE
↓
Register viewer presence in Redis
↓
Create/update viewer session if required
↓
Return stream playback information
```

The endpoint must not expose private stream secrets.

24. Viewer Presence

Redis key:

```
stream:{streamId}:viewers
```

Use a Set or equivalent structure.

Example:

```
user_a  
user_b  
user_c
```

Count:

```
SCARD
```

Viewer heartbeat:

```
every 15-30 seconds
```

Stale viewers expire.

On disconnect:

```
remove presence
```

25. Viewer Count Consistency

Viewer count is ephemeral.

Redis is authoritative for the real-time count.

PostgreSQL stores durable stream information.

If Redis becomes unavailable:

- Do not corrupt PostgreSQL.
- Do not fabricate counts.
- Degrade gracefully.
- Log the infrastructure error.
- Recover presence when Redis returns.

The stream itself should not be ended merely because a viewer-count update fails.

26. End Stream

```
POST /streams/:id/end
```

Allowed:

```
stream owner  
ADMIN
```

Process:

```
Authenticate
↓
Authorize
↓
Check stream
↓
Validate current state
↓
Stop/close media publishing
↓
BEGIN DB TRANSACTION
↓
set status = ENDED
↓
set ended_at
↓
finalize required stream state
↓
COMMIT
↓
clear Redis presence
↓
broadcast stream ended event
```

If the database transaction fails:

```
ROLLBACK
```

The system must not falsely report the stream as permanently ended.

27. Chat

Use Socket.IO.

Required events:

```
stream:join
stream:leave
chat:send
chat:message
```

Additional:

```
viewer:joined
viewer:left
gift:sent
stream:status
```

28. Socket Authentication

The Socket.IO connection must be authenticated.

Do not allow anonymous clients to impersonate users.

Server resolves:

```
socket.user.id
```

```
socket.user.role
```

Never trust:

```
userId
```

from event payloads.

29. Chat Flow

```
chat:send
↓
Authenticate socket
↓
Validate stream
↓
Validate message
↓
Check membership/presence
↓
Rate limit
↓
Persist message
↓
Emit chat:message
```

If persistence fails:

- Do not broadcast the message as successfully persisted.
- Return a structured error.
- Client can retry if appropriate.

30. Chat Database

```
chat_messages
```

```
id UUID
stream_id UUID
user_id UUID
message TEXT
created_at TIMESTAMP
deleted_at TIMESTAMP NULL
```

Index:

```
(stream_id, created_at)
```

31. Chat Safety

Validate:

```
minimum length
maximum length
UTF-8 validity
```

whitespace

Rate limit per authenticated user.

Example:

5 messages / 10 seconds

Repeated violations may result in temporary chat restrictions.

32. Wallet

Required:

GET /wallet

Wallet is server-owned.

Client cannot submit:

balance

as an authoritative value.

Database:

wallets

```
id UUID
user_id UUID UNIQUE
balance BIGINT
version BIGINT
created_at
updated_at
```

Credits must be integer values.

Never use floating-point numbers.

33. Gift Catalog

Required:

GET /gifts

Seed:

Rose	10
Heart	25
Star	50
Diamond	100

Gift prices come from the database.

Frontend-provided prices are ignored.

34. Send Gift

Required:

```
POST /streams/:id/gifts
```

Request:

```
{
  "giftId": "gift_uuid",
  "quantity": 1,
  "idempotencyKey": "client-generated-unique-key"
}
```

The client does not send:

```
price
senderId
receiverId
```

Backend determines all of these.

35. Gift Transaction — Critical Safety Requirement

The complete operation must be a PostgreSQL transaction.

```
BEGIN
↓
validate authenticated sender
↓
validate stream
↓
validate broadcaster
↓
load gift from database
↓
lock sender wallet
↓
calculate price from DB
↓
check balance
↓
lock/check idempotency record
↓
deduct sender balance
↓
create gift transaction
↓
COMMIT
```

If ANY operation fails:

```
ROLLBACK EVERYTHING
```

No partial wallet deduction.

36. Concurrent Gift Protection

Example:

```
Balance = 100
Gift = 100
```

Two simultaneous requests arrive.

Use row-level locking/appropriate PostgreSQL transaction isolation so only one can successfully spend the credits.

The second request must observe the updated balance and fail safely.

This prevents negative balances and race conditions.

37. Gift Transaction Database

```
gift_transactions

id UUID
idempotency_key VARCHAR
sender_id UUID
receiver_id UUID
stream_id UUID
gift_id UUID
quantity INTEGER
unit_cost BIGINT
total_cost BIGINT
created_at TIMESTAMP
```

Unique constraint:

```
UNIQUE(sender_id, idempotency_key)
```

38. Idempotency

If a client retries:

```
POST /streams/:id/gifts
```

with the same idempotency key:

- Do not charge twice.
- Return the original transaction/result.

This protects against:

- Network retries.
- Browser retries.
- Mobile retries.
- Timeout followed by client retry.

39. Gift Event Ordering

Only emit:

```
gift:sent
```

after the database transaction commits successfully.

Never:

```
emit gift
↓
database transaction
```

because the database could roll back after viewers already saw a gift that never actually happened.

Correct:

```
DB COMMIT
↓
emit event
```

For stronger reliability, an outbox pattern can later be introduced.

40. Outbox-Ready Design

For critical domain events:

```
gift sent
stream ended
user disabled
```

we should design so an outbox can be added without changing the public API.

Future:

```
BEGIN
  update business data
  insert outbox event
COMMIT

worker
↓
publish event
↓
mark outbox processed
```

For the assessment MVP, Socket.IO events can be emitted after successful commits.

41. Reports

Add:

```
POST /reports
GET /reports
GET /reports/:id
PATCH /reports/:id
```

Report target types:

```
USER
STREAM
CHAT_MESSAGE
```

Statuses:

```
OPEN
REVIEWING
RESOLVED
DISMISSED
```

42. Admin

Required:

```
/admin
```

APIs:

```
GET /admin/users
GET /admin/streams
GET /admin/reports

PATCH /admin/users/:id/status
POST /admin/streams/:id/end
```

All require:

```
authenticated
+
ADMIN
```

43. Disabling Users

When a user is disabled:

- Login must fail.
- Existing API access must be rejected.
- Socket authentication must reject the user.
- Active stream should be terminated safely.
- User should not be able to send chat.
- User should not be able to send gifts.

This should be implemented as an explicit administrative workflow.

44. Database Transactions — Global Rule

Any operation involving multiple related writes must use a transaction.

Examples:

```
registration
follow + follower counts
stream finalization
gift purchase
admin user disable + dependent state
```

Rule:

```
ALL OR NOTHING
```

If operation fails halfway:

```
ROLLBACK
```

No partial state.

45. Failure-Safe Rules

Database failure

- Return controlled error.
- Roll back active transaction.
- Never return success.
- Log request ID and error.
- Do not leak database details.

Redis failure

- Persistent operations continue where safe.
- Real-time presence may temporarily degrade.
- Do not fabricate authoritative financial state.
- Retry only safe/idempotent operations.
- Do not block unrelated PostgreSQL operations unnecessarily.

Socket failure

- Client reconnects.
- Authentication is revalidated.
- Stream presence is restored.
- Duplicate joins must be harmless.

Media server failure

- Stream status must not blindly remain LIVE forever.
- Health/heartbeat logic should detect stale streams.
- Admin/backend can finalize abandoned sessions.

Client timeout

- Client retries only idempotent or explicitly idempotent operations.
- Gift requests always require idempotency keys.

46. Health Checks

```
GET /health/live
GET /health/ready
```

Live:

```
application process is running
```

Ready:

```
PostgreSQL available
Redis available
required infrastructure available
```

Do not expose secrets or internal credentials.

47. Security Requirements

HTTP

Use:

- Helmet.
- Strict CORS.
- Request body limits.
- Secure headers.
- HTTPS in deployment.
- Input validation.

Authentication

- Strong password hashing.
- Short-lived access tokens.
- Refresh-token rotation.
- Secure cookies.
- Token revocation/reuse detection.

Authorization

Every protected resource checks ownership/role.

Input validation

Validate:

- Body.
- Query parameters.
- Route parameters.
- UUIDs.
- Titles.
- Chat messages.
- Gift quantities.

48. Rate Limiting

Different endpoints have different limits.

Example baseline:

```
Login:      5/min/IP
Register:   5/hour/IP
General API: 100/min/user
Chat:       5/10sec/user
Gift:       10/min/user
```

These should be configurable.

Redis-backed rate limiting should be used so limits work across multiple API instances.

49. SQL/Data Integrity

Database constraints must enforce important invariants.

Examples:

```
UNIQUE users.email
```

```
UNIQUE users.username
UNIQUE profiles.user_id
UNIQUE follows(follower_id, following_id)
UNIQUE wallets.user_id
UNIQUE gift_transactions(sender_id, idempotency_key)
```

Use foreign keys.

Use appropriate ON DELETE behavior deliberately.

Do not rely only on application code for integrity.

50. Pagination

Never return unbounded collections.

Use pagination for:

```
users
streams
live streams
chat history
gift history
reports
```

Preferred long-term strategy:

```
cursor pagination
```

Example:

```
GET /streams/live?limit=20&cursor=...
```

51. API Validation

Use centralized schemas.

Example:

```
CreateStreamSchema
LoginSchema
RegisterSchema
SendMessageSchema
SendGiftSchema
UpdateProfileSchema
ReportSchema
```

Validation happens before business logic.

52. Error Taxonomy

Use stable application error codes:

```
AUTH_INVALID_CREDENTIALS
AUTH_TOKEN_EXPIRED
AUTH_FORBIDDEN

USER_NOT_FOUND
USER_DISABLED

STREAM_NOT_FOUND
STREAM_NOT_LIVE
STREAM_ALREADY_LIVE
STREAM_NOT_OWNER
STREAM_INVALID_STATE

CHAT_INVALID_MESSAGE
CHAT_RATE_LIMITED

GIFT_NOT_FOUND
GIFT_INVALID_QUANTITY
INSUFFICIENT_BALANCE
GIFT_DUPLICATE_TRANSACTION

REPORT_NOT_FOUND

ADMIN_REQUIRED
```

Frontend should use codes rather than matching human-readable messages.

53. Frontend Pages

Required:

```
/login
/register
/
/stream/:id
/profile/:id
/admin
```

54. Home

Display:

- Live streams.
- Broadcaster.
- Stream title.
- Viewer count.

Use:

```
GET /streams/live
```

Add loading, empty, and error states.

55. Live Stream Page

Required:

Video
Broadcaster
Viewer count
Chat
Gift controls
Follow

The page must handle:

stream loading
stream ended
media connection failed
socket disconnected
socket reconnecting
insufficient credits
gift failure

56. Profile

Display:

- Profile picture.
- Username.
- Display name.
- Bio.
- Followers.
- Following.
- Follow/unfollow.
- Live status.

57. Admin UI

Dashboard:

Total Users
Active Streams
Total Streams
Active Users
Reports

Users:

User
Username
Status
Created Date

Action

Streams:

Broadcaster
Title
Viewers
Started
Status
Action

Reports:

Target
Reporter
Reason
Status
Created
Action

58. WebSocket Reliability

Client behavior:

CONNECT
↓
AUTHENTICATE
↓
JOIN STREAM
↓
HEARTBEAT
↓
RECEIVE EVENTS

On disconnect:

reconnect
↓
re-authenticate
↓
rejoin stream
↓
restore presence

Events must be safe to receive more than once where possible.

59. Media Failure Handling

If WebRTC fails:

Frontend displays:

Unable to connect to live video.
Retry

Do not crash the entire page.

Chat should continue if media fails.

If chat fails:

Video should continue if possible.

This means video, chat, and API functionality must be loosely coupled.

60. Logging

Use structured logs.

Every request should include:

```
requestId
method
route
status
duration
userId
```

Never log:

```
password
access token
refresh token
wallet secrets
stream credentials
```

61. Observability

MVP:

- Structured logging.
- Request IDs.
- Health endpoints.
- Error logging.
- Database error monitoring.

Future:

- OpenTelemetry.
- Metrics.
- Distributed tracing.
- Prometheus/Grafana.
- Centralized logs.

62. Testing

Minimum requirement: 10 meaningful automated tests.

Target:

20+ tests

Authentication

- Register.
- Duplicate email.
- Login.
- Invalid password.
- Unauthorized API request.

Streaming

- Create stream.
- Prevent duplicate live stream.
- Join stream.
- End own stream.
- Reject unauthorized end.
- Admin can end stream.

Chat

- Join.
- Send.
- Receive.
- Invalid message.
- Rate limit.

Wallet/Gifts

- Wallet retrieval.
- Gift catalog.
- Successful gift.
- Insufficient balance.
- Invalid gift.
- Duplicate transaction.
- Concurrent gift protection.

63. Transaction Tests

Explicitly test rollback.

Example:

```
BEGIN
wallet deduction
gift transaction creation fails
ROLLBACK
```

Expected:

```
wallet unchanged
gift transaction absent
```

This is a critical acceptance test.

64. Concurrency Tests

Test:

```
Two simultaneous gifts
same wallet
same balance
```

Expected:

```
No negative balance
No double spending
Exactly valid successful transactions
```

65. API Documentation

Swagger:

```
/api/docs
```

Document:

- Authentication.
- Users.
- Profiles.
- Streams.
- Chat.
- Gifts.
- Wallet.
- Admin.
- Reports.

Every endpoint includes:

- Description.
- Auth requirement.
- Request.
- Response.
- Error responses.
- Example payload.

66. Database Migration Strategy

Prisma migrations:

```
prisma migrate dev
prisma migrate deploy
```

Never manually modify production schema.

Every schema change must be represented by a migration.

Seeds should create:

```
admin
test users
gift catalog
test wallet credits
```

67. Seed Data

Example:

```
Admin
admin@example.com

Broadcaster
broadcaster@example.com

Viewer
viewer@example.com
```

Wallet:

```
1000 credits
```

Gifts:

```
Rose 10
Heart 25
Star 50
Diamond 100
```

No real secrets in repository.

68. Development Without Docker

Initial local setup:

```
Next.js
Node.js
PostgreSQL
Redis
MediaMTX
```

PostgreSQL and Redis can be installed/run locally or through their native installers/services.

Docker is deliberately excluded from the first implementation.

69. Git Strategy

Branches:

```
main
develop
feature/*
fix/*
```

Commits should be focused:

```
feat(auth): implement registration
feat(streams): add stream lifecycle
feat(chat): add socket chat
fix(wallet): prevent concurrent overspending
test(gifts): add transaction rollback tests
```

70. Definition of Done

A feature is not complete merely because the happy path works.

For every feature:

```
Happy path
+
Validation
+
Authorization
+
Failure handling
+
Transaction safety where required
+
Tests
+
Documentation
```

Example:

Gift feature is not complete until:

success
invalid gift
insufficient balance
duplicate request
concurrent requests
database failure
rollback
unauthorized user
stream ended

are handled.

71. Scalability

~100 users

Single backend instance is sufficient.

Next.js
Express
PostgreSQL
Redis
MediaMTX

~10,000 users

Introduce:

Load balancer
Multiple API instances
Redis shared infrastructure
PostgreSQL connection pooling
Dedicated media infrastructure

100,000+

Introduce:

Multiple regions
Dedicated media clusters
CDN/edge delivery
Database replicas
Partitioning where necessary
Distributed observability
Autoscaling
Dedicated realtime infrastructure

The assessment specifically asks for this scalability reasoning rather than requiring actual 100,000-user support.

72. Critical Architecture Principle

Persistent state:

PostgreSQL

Ephemeral state:

Redis

Video:

Media Server

Realtime:

Socket.IO

Business logic:

Express Services

UI:

Next.js

Never mix these responsibilities unnecessarily.

73. Failure-Safety Principle

The platform follows:

```
Validate
↓
Authorize
↓
Execute atomically
↓
Commit
↓
Publish event
```

Never:

```
Publish event
↓
Try database operation
```

For transactional operations:

```
BEGIN
↓
all writes
↓
success?
■■■■■
YES NO
■ ■
COMMIT ROLLBACK
■
publish event
```

This is mandatory for wallet/gift operations and any multi-write business operation.

74. Final API Contract

Auth

```
POST /auth/register
POST /auth/login
POST /auth/logout
```

Users

```
GET    /users/me
GET    /users/:id
PUT    /users/me
POST   /users/:id/follow
DELETE /users/:id/follow
```

Streams

```
POST /streams
GET  /streams
GET  /streams/live
GET  /streams/:id
POST /streams/:id/join
POST /streams/:id/end
```

Wallet/Gifts

```
GET /wallet
GET /gifts
POST /streams/:id/gifts
GET /users/:id/gifts
```

Reports

```
POST /reports
GET  /reports
GET  /reports/:id
PATCH /reports/:id
```

Admin

```
GET  /admin/users
GET  /admin/streams
GET  /admin/reports
PATCH /admin/users/:id/status
POST  /admin/streams/:id/end
```

Health

```
GET /health/live
GET /health/ready
```

75. WebSocket Contract

```
stream:join
stream:leave

chat:send
chat:message

viewer:joined
viewer:left

gift:sent

stream:status
```

76. Required Deliverables

Repository must contain:

```
Source code
README.md
.env.example
Prisma schema
Migrations
Seed
Swagger
ER diagram
Architecture diagram
Streaming architecture documentation
Security documentation
Automated tests
```

The assessment explicitly requires source code, README, database artifacts, Swagger, architecture documentation, and a working demo if possible.

77. Final Acceptance Criteria

The system is accepted when:

Authentication

- Users can register/login/logout.
- Passwords are securely hashed.
- Unauthorized requests fail.
- Admin APIs require ADMIN.

Profiles

- Users can manage profiles.
- Users can follow/unfollow.
- Duplicate/self follows are prevented.

Streaming

- User can create a stream.

- User can publish live video.
- Viewer can join and watch.
- Viewer count works.
- Owner can end stream.
- Admin can end stream.
- Unauthorized users cannot terminate streams.

Chat

- Users can join chat.
- Messages arrive in real time.
- Messages persist.
- Validation and rate limiting work.
- Socket reconnect works.

Gifts

- Users have virtual credits.
- Gifts use server-defined prices.
- Gift transactions are atomic.
- Insufficient balances fail.
- Duplicate requests do not double charge.
- Concurrent requests cannot overspend.
- Failed transactions roll back completely.

Admin

- Users can be viewed/disabled.
- Streams can be viewed/terminated.
- Reports can be reviewed.

Engineering

- PostgreSQL migrations exist.
- Redis is used appropriately.
- Swagger exists.
- Tests exist.
- Errors are consistent.
- Logs are structured.
- No secrets are committed.
- No Docker dependency exists initially.
- README allows a developer to reproduce the project.

78. Implementation Priority

Build in this order:

1. Repository + TypeScript foundation
2. PostgreSQL + Prisma
3. Redis
4. Express architecture
5. Authentication
6. Users + profiles
7. Follow system
8. Stream lifecycle
9. Media/WebRTC integration
10. Viewer presence
11. Socket.IO chat
12. Wallet
13. Gifts + transactions
14. Reports
15. Admin
16. Frontend polish
17. Automated tests
18. Swagger
19. Security hardening
20. Documentation

79. Engineering Standard

The implementation should optimize for:

Correctness
Security
Atomicity
Failure recovery
Clear boundaries
Testability
Maintainability
Observability
Scalability

rather than simply maximizing feature count.

The result should look like a **small production system**, not a CRUD assessment project.